Unified Conversational Dashboard Technical Background

Scope

This document explains the Unified Conversational Dashboard as the AI-powered conversational layer of the ElderMeds system. It focuses on how the assistant queries across system data, generates answers, and performs safe agentic actions.

This document does not focus on the visual dashboard summary cards. It focuses on the conversational assistant: the part that accepts natural language, retrieves health records, explains results, and safely performs confirmed changes.

Purpose

The Unified Conversational Dashboard allows an elder or caregiver to ask health-related questions in natural language instead of navigating many screens manually.

Example questions:

```
Did I miss any medicine this week?
Which medicines are running low?
How has my mood been recently?
Are any medicines dangerous for me?
Show my recent caregiver alerts.
How can I reduce this diabetes risk?
Add 20 tablets to Metformin stock.
Change breakfast routine to 8 AM.
```

The assistant can:

- Query medication, routine, allergy, mood, caregiver alert, and prediction records.
- Generate simple conversational answers.
- Provide follow-up questions.
- Use voice input and text-to-speech output.
- Save conversation history.
- Trigger health-risk prediction workflows.
- Give trusted health-risk advice.
- Save emotional support check-ins.
- Perform confirmed agentic actions such as updating stock or routine times.

Technologies Used

Frontend technologies:

- React Native for the mobile application UI.
- Expo for app runtime and native module integration.
- Axios for HTTP communication with the backend.
- expo-speech for text-to-speech output.
- expo-speech-recognition for voice input.

Backend technologies:

- Node.js runtime.
- Express.js API server.
- PostgreSQL database.
- pg Node driver for SQL queries.
- JWT authentication for user identity.
- Custom SQL validation for LLM-generated SQL.
- Deterministic agentic action parser and executor.

ML service technologies:

- Python.
- FastAPI.
- Pydantic request/response schemas.
- spaCy preprocessing for text normalization, keywords, entities, and time hints.
- Ollama as the local LLM runtime interface.
- Llama 3 style chat model for NL2SQL and answer generation.

Data technologies:

- PostgreSQL relational schema.
- JSONB fields for stored retrieved rows and action metadata.
- Health advice knowledge chunks for trusted RAG answers.

Main System Components

Frontend components:

```
frontend/src/screens/AssistantChatScreen.js
frontend/src/services/assistantService.js
frontend/src/components/ChatBubble.js
frontend/src/components/AssistantFAB.js
frontend/App.js
```

Backend components:

```
backend/src/routes/assistant.js
backend/src/controllers/assistantController.js
backend/src/services/assistantOrchestrator.js
backend/src/services/sqlValidator.js
backend/src/data/schemaDescription.js
backend/src/services/assistantAgenticActionService.js
backend/src/services/healthAdviceService.js
```

ML service components:

```
ml-service/app/routers/assistant.py
ml-service/app/services/nl2sql.py
ml-service/app/services/rag_answer.py
ml-service/app/services/health_advice_answer.py
ml-service/app/services/spacy_preprocess.py
ml-service/app/services/ollama_client.py
```

Database migration:

```
backend/migrations/1746590400000_assistant_tables.js
```

High-Level Architecture

The assistant is designed as a controlled orchestration system. The LLM helps with language understanding and explanation, but the backend controls database access and all data-changing operations.

High-level flow:

```
User message
  -> React Native assistant screen
  -> Node/Express backend
  -> assistant orchestrator
  -> route intent
  -> optional ML service call
  -> optional PostgreSQL query
  -> optional action confirmation/execution
  -> save conversation history
  -> return final response to frontend
```

The key design principle is:

```
The LLM may suggest SQL or generate text, but it never directly executes database op
erations.
```

Frontend Flow

The assistant screen is AssistantChatScreen.js.

When the user sends a message, the screen calls:

```
assistantService.chat({
  message,
  conversationId
})
```

That sends:

```
POST /api/assistant/chat
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "message": "Which medicines are running low?",
  "conversationId": 12
}
```

The frontend keeps local state for:

- messages
- input
- conversationId
- sending

- error
- isListening
- speakingId
- chat history modal state
- rename/delete conversation state
- follow-up prompt state

The assistant can also receive an initialPrompt. This is used when another screen opens the assistant with a pre-filled task, such as asking for a risk summary or reviewing alerts. The chat screen automatically sends the initial prompt once.

Backend API Surface

The assistant API is mounted under:

```
/api/assistant
```

Main endpoints:

```
POST   /api/assistant/chat
GET    /api/assistant/conversations
GET    /api/assistant/conversations/:id/messages
PATCH  /api/assistant/conversations/:id
DELETE /api/assistant/conversations/:id
```

The chat endpoint is protected by:

- database availability middleware
- JWT authentication middleware
- per-user chat rate limiting

The default chat rate limit is configured by:

```
ASSISTANT_RATE_LIMIT_PER_MIN
```

Assistant Orchestrator

The central backend component is:

```
backend/src/services/assistantOrchestrator.js
```

It receives:

```
{
  userId,
  role,
  message,
  conversationId
}
```

It performs routing in this order:

1. Check whether the message confirms or cancels a pending agentic action.

2. Parse a new agentic action request.
3. Detect emotional support check-in requests.
4. Detect trusted health-risk advice questions.
5. Detect stroke risk prediction requests.
6. Detect hypertension risk prediction requests.
7. Detect diabetes risk prediction requests.
8. Fall back to the general NL2SQL + RAG database question-answering flow.

This routing is important because not every message should go through generic SQL generation. Some workflows are safer and more accurate when handled deterministically.

General Database Question Answering

For normal health-record questions, the assistant uses a two-stage RAG pipeline.

Example:

```
User: Did I miss any medicine this week?
```

Flow:

```
1. Frontend sends the message to POST /api/assistant/chat.
2. Backend authenticates the user.
3. Backend ensures a conversation exists.
4. Backend loads recent conversation messages for context.
5. Backend builds a schema digest from schemaDescription.js.
6. Backend calls ML service /assistant/nl2sql.
7. ML service returns SQL and intent.
8. Backend validates the SQL.
9. Backend executes the SQL with $1 = authenticated user id.
10. Backend redacts sensitive fields.
11. Backend calls ML service /assistant/answer.
12. ML service generates a conversational answer.
13. Backend saves user and assistant messages.
14. Frontend displays the answer.
```

NL2SQL Stage

The first ML stage is:

```
POST /assistant/nl2sql
```

Implemented in:

```
ml-service/app/services/nl2sql.py
```

Input includes:

- user question
- user role
- recent messages
- schema digest
- spaCy preprocessing hints

Output shape:

```
{
  "sql": "SELECT ... WHERE user_id = $1 LIMIT 50",
  "params": ["$1"],
  "intent": "missed_doses_7d",
  "fallback": false,
  "fallback_message": ""
}
```

The NL2SQL service uses two strategies:

1. Rule-based SQL for common query patterns.
2. Llama 3 NL2SQL generation for more flexible queries.

Rule-based SQL improves reliability for high-value flows such as:

- missed doses
- medication schedule
- routine lookup
- allergy profile lookup
- medicine safety lookup
- mood trends
- caregiver alerts
- low stock
- current medications
- cognitive activity history

Schema Digest

The backend does not give the LLM unrestricted database visibility. Instead, it builds a schema digest from:

```
backend/src/data/schemaDescription.js
```

The schema digest describes:

- allowed tables
- allowed columns
- user scoping rules
- safe relationships
- hard domain rules

Sensitive fields are intentionally omitted or marked as forbidden.

The assistant can query across these broad areas:

- user account profile
- daily routine
- allergy profile
- medicine safety checks
- reaction logs

- current medications
- medication stock
- medication status events
- caregiver alerts
- health profiles
- diabetes, stroke, and hypertension predictions
- emotional support sessions
- emotional interventions
- cognitive activity attempts
- emotional caregiver alerts
- trusted health advice chunks

SQL Validation

Generated SQL is validated by:

```
backend/src/services/sqlValidator.js
```

The validator allows:

- exactly one statement
- SELECT-only queries
- only tables listed in schemaDescription.js
- mandatory user scoping for user-specific tables
- only $1 as a parameter
- a hard result limit

The validator blocks:

- INSERT
- UPDATE
- DELETE
- DROP
- ALTER
- CREATE
- COPY
- comments
- multiple statements
- information_schema access
- pg_* catalog access
- unscoped access to user data
- extra parameters such as $2

After validation, the backend executes the query with:

```
$1 = current authenticated user id
```

This prevents the LLM from reading another user’s data.

SQL Execution

SQL execution happens only in the Node backend.

The backend:

- opens a PostgreSQL client
- starts a transaction
- sets a local statement timeout
- executes the validated query
- commits on success
- rolls back on failure
- returns at most the configured maximum rows

Important configuration:

```
ASSISTANT_SQL_TIMEOUT_MS
ASSISTANT_MAX_ROWS
```

Default behavior:

- statement timeout around 5 seconds
- maximum returned rows around 100

Redaction Before LLM Answering

Before database rows are sent to the ML answer generator, sensitive values are redacted.

Sensitive keys include:

```
password
password_hash
token
authorization
email
phone
caregiver_email
caregiver_phone
```

This means the final answer model receives only the data required for answering the health question.

RAG Answer Generation Stage

The second ML stage is:

```
POST /assistant/answer
```

Implemented in:

```
ml-service/app/services/rag_answer.py
```

Input includes:

- original question
- SQL that was safely executed
- retrieved rows
- intent
- user role

The answer generator returns:

```
{
  "answer": "You missed 2 doses this week...",
  "follow_ups": [
    "Show me the missed doses.",
    "Which medicines are running low?"
  ]
}
```

The answer is grounded in the retrieved rows. If no rows are found, the assistant gives a no-matching-records response rather than inventing data.

Conversation Persistence

The assistant stores conversation history in PostgreSQL.

Tables:

```
assistant_conversations(
  id,
  user_id,
  title,
  created_at,
  updated_at
)

assistant_messages(
  id,
  conversation_id,
  user_id,
  role,
  content,
  sql_used,
  rows_returned,
  intent,
  fallback_reason,
  latency_ms,
  created_at
)
```

The stored data supports:

- loading previous chats
- reopening one conversation
- preserving short-term context
- renaming conversations
- deleting conversations
- auditing SQL and fallback reasons
- measuring latency

- storing pending and completed action metadata

Agentic Features

Agentic features are features where the assistant can perform an application action, not only answer a question.

Supported agentic domains:

- medication
- medicine stock
- daily routine
- caregiver alerts

Supported actions include:

- create a medicine
- update a medicine
- delete a medicine
- add stock to a saved medicine
- set current stock quantity
- change breakfast, lunch, dinner, or sleep routine time
- create refill alerts
- create low-stock alerts
- mark caregiver alerts as read
- delete caregiver alerts

Agentic Action Flow

Agentic actions use a confirmation-first design.

Example:

```
User: Add 20 tablets to Metformin stock.
Assistant: Add 20 tablets to Metformin stock. Please confirm before I make this chan
ge. Reply "yes" to continue or "cancel" to stop.
User: yes
Assistant: Added 20 tablets to Metformin stock. Current stock is 45. I will take you
 to the affected screen now.
```

Technical flow:

```
1. User sends an action-like command.
2. Backend parses it using deterministic rules.
3. Backend creates a pending action object.
4. Backend stores the pending action in assistant_messages.rows_returned.
5. Assistant asks for confirmation.
6. User replies yes or cancel.
7. Backend retrieves the latest pending action.
8. If cancelled, no database write happens.
9. If confirmed, backend executes the correct action handler.
10. Backend saves result metadata.
11. Frontend may navigate to the affected screen.
```

The LLM is not used to directly execute these writes. The backend performs the write using controlled service functions.

Agentic Action Parsing

Agentic action parsing is implemented in:

```
backend/src/services/assistantAgenticActionService.js
```

The parser converts text into structured action objects.

Example:

```
Add 20 tablets to Metformin stock
```

becomes:

```
{
  domain: "stock",
  operation: "refill_stock",
  target: {
    medicineName: "Metformin"
  },
  payload: {
    quantity: 20
  }
}
```

The system also handles:

- number words such as "twenty"
- medicine synonym matching
- routine time parsing
- target medicine lookup
- missing-detail responses
- medication not found responses with available medicine names

Agentic Action Execution

After confirmation, actions are executed by backend functions such as:

- createMedication
- updateMedication
- deleteMedication
- updateRoutine
- refillStock
- createCaregiverAlert
- updateCaregiverAlert

Every action is scoped by authenticated userId.

For example, stock updates use:

```
WHERE medication_id = $1 AND user_id = $2
```

This prevents a user or caregiver from changing another user's records.

Frontend Navigation After Actions

Some completed actions return navigation metadata.

Example:

```
{
  screen: "home",
  launchIntent: {
    type: "medicine-stock",
    highlight: {
      kind: "medicationName",
      value: "Metformin"
    }
  }
}
```

The assistant screen reads this metadata and calls the app-level navigation handler. It may speak the answer first, then navigate after a short delay.

Health Risk Prediction Integration

The assistant can detect requests for:

- diabetes risk
- stroke risk
- hypertension risk

Example messages:

```
Check my diabetes risk.
Check my stroke risk.
Check my hypertension risk.
My BP is 145 over 90.
Now my weight is 60kg.
```

These are handled by deterministic backend prediction services instead of generic NL2SQL.

The prediction services:

- extract health values from the message
- combine them with saved health profile data
- call the relevant prediction workflow
- save prediction history
- return risk level, confidence, algorithm, factors, and summary

The assistant then returns the prediction summary as a chat response with follow-up prompts.

Trusted Health Advice RAG

Health advice uses a separate RAG path from the general database query flow.

Example questions:

```
How can I reduce this risk?
Why is my diabetes risk high?
What should my caregiver monitor?
```

Flow:

```
1. Backend detects health advice intent.
2. Backend loads latest diabetes, stroke, or hypertension prediction.
3. Backend selects the relevant risk type.
4. Backend retrieves trusted health advice chunks.
5. Backend sends prediction context and chunks to ML service.
6. ML service generates a short answer using only those chunks.
7. Backend stores the answer and source metadata.
8. Frontend displays the answer and trusted source names.
```

This path supports:

- diabetes advice
- hypertension advice
- stroke advice

It intentionally refuses unsupported advice domains in this phase.

Emotional Support Integration

The assistant can also detect emotional-support messages.

Examples:

```
I feel lonely today.
I feel confused and need support.
Has my mother seemed lonely recently?
```

The backend runs:

```
analyzeCheckIn()
  -> calculateRiskLevel()
  -> selectIntervention()
  -> getNextActivityByEmotion()
  -> createCheckInRecord()
  -> optionally create caregiver alerts
```

The assistant response can include:

- detected emotion
- confidence
- stress score
- loneliness score
- risk level
- supportive chatbot reply
- recommended cognitive activity
- caregiver alert creation status

Voice Input and Voice Output

Voice input:

```
User taps microphone
  -> expo-speech-recognition requests permission
  -> speech is transcribed
  -> transcript fills the input
  -> final transcript is sent as a chat message
```

Voice output:

```
User taps Listen
  -> expo-speech reads the assistant response aloud
  -> user can stop playback
```

If speech recognition is not available in the app build, the assistant falls back to typed input.

Caregiver Mode

The assistant supports elder and caregiver roles.

Caregiver mode changes phrasing and suggested prompts. For example:

```
How is your mother doing today?
Which medicines are running low for your elder?
What should I monitor as a caregiver?
```

The important security point is that caregiver access still resolves to the elder's user id. Queries still use:

```
WHERE user_id = $1
```

or the correct equivalent elder-scoped column such as:

```
WHERE elder_user_id = $1
```

Fallback Handling

The assistant has several fallback paths:

- Empty message returns a validation error.
- Out-of-scope NL2SQL returns a polite limitation message.
- ML service unreachable returns an AI-unavailable message.
- SQL validation failure returns a safe refusal.
- SQL execution failure returns a database-query failure message.
- Answer generation failure returns a simpler fallback based on whether rows were found.
- Missing agentic action details returns a request for more information.
- Failed confirmed action returns a specific explanation when possible.

Fallback reasons are stored in assistant_messages.fallback_reason.

Security Model

Security controls include:

- JWT authentication on every assistant endpoint.
- Authenticated userId is the only database scope parameter.
- LLM-generated SQL is SELECT-only.
- SQL is validated before execution.
- User-scoped tables must include user scoping.
- Only allowed tables from schemaDescription.js can be queried.
- Only $1 parameter is allowed.
- Database timeout is enforced.
- Row limits are enforced.
- Sensitive fields are redacted before answer generation.
- Data-changing actions require explicit user confirmation.
- Agentic writes are performed by backend code, not by LLM-generated SQL.
- Chat requests are rate-limited per user.

Why This Design Is Appropriate

The system needs natural language flexibility, but it also handles sensitive healthcare records. The architecture therefore separates language intelligence from authority.

The LLM can:

- interpret user questions
- generate candidate SQL
- summarize retrieved records
- produce friendly answers
- suggest follow-up questions

The backend controls:

- authentication
- database access
- SQL validation
- user scoping
- action parsing
- write execution
- conversation persistence
- fallback handling

This gives the user a conversational experience while preserving strong control over data access and modifications.

End-to-End Summary

The Unified Conversational Dashboard is an AI-assisted healthcare control layer built with

React Native, Node/Express, PostgreSQL, FastAPI, spaCy, Ollama, and Llama 3.

Its central function is to let users ask natural-language questions across the ElderMeds system and receive grounded answers based on their own records.

Its agentic function is to safely perform selected app actions only after confirmation.

The core technical pattern is:

```
Natural language
  -> intent routing
  -> safe retrieval or confirmed action
  -> grounded conversational answer
  -> persisted chat history
```

This makes the dashboard unified because the same assistant interface can answer questions about medicines, adherence, stock, allergies, routines, mood, caregiver alerts, risk predictions, and trusted health advice, while also safely completing selected user-approved tasks.